User Preference-Based Hierarchical Offloading for Collaborative Cloud-Edge Computing

Shujuan Tian, Chi Chang, Saiqin Long, Sangyoon Oh, Zhetao Li, *Member, IEEE*, and Jun Long

Abstract—Cloud computing and mobile edge computing techniques supply efficient ways to solve the contradiction between the increasing computing and storage demands of portable terminals and the limited capacity. In this paper, we conduct a three-tier hierarchical service system with multiple mobile users(UEs), multiple mobile edge computing servers(MECs), and a single cloud center (CC). It is worth noting that multiple UEs with personalized options generate a large number of different tasks in real time. To deal with this complex offloading problem, a response ratio offloading strategy (RROS) centered on user preference and real-time nature is designed to make MECs or CC serve as many UEs as possible. Therefore, a MEC-choosing preference list of each UE is created based on its past experiences at first. Then, each MEC iteratively sorts UEs with its ranking in the UEs' preference list. In order to avoid that the first task arriving at MEC occupies too many resources of MEC and cannot achieve global optimization, we also adopt loop iterative sequencing for multiple tasks arriving within a stipulated time. Lastly, by comparing the optimal response ratio on different MECs and CC, multiple MECs and the CC collaborative offload computing tasks of multiple UEs. We demonstrate numerical examples and data of the proposed strategy. Experimental results show that the algorithm significantly outperforms conventional techniques even with the increase of users.

Index Terms—Offloading strategy optimization, mobile edge computing, hierarchy, average response time, energy consumption

1 Introduction

1.1 Motivation

Except for the existing relatively mature smartphones, more and more wearable devices, sensors and other mobile and Internet of things (IoT) devices have become very popular in modern society [1]. The development of mobile and intelligent IoT devices promotes the diversity and complexity of application software, such as face recognition [2], natural language processing [3], intelligent traffic navigation [4], etc. These applications usually require high processing power, high memory/storage, and high energy consumption. Due to mobile/IoT devices are limited by size, battery lifetime, heat dissipation and resources, applications running on resource-poor mobile/IoT devices will fail to execute or have a poor user experience [5]. To meet the growing performance requirements of users and the scarcity of mobile/IoT device resources, cloud providers provide their cloud services for mobile/IoT devices to alleviate this problem.

Cloud computing has many advantages over traditional local physical deployment, such as lower infrastructure costs, rich computing/storage resources and flexible resource allocation [6]. The cloud platform serves a carrier of user task. It provides a cloud service center to share cloud resources to users [7]. Users can offload complex and heavy tasks to the cloud computing center to complete the computing. Cloud computing centers can ensure the execution speed of complex applications, computing accuracy, and the computing energy consumption of user terminal devices. However, due to the long distance between the cloud computing center and users, the centralized network architecture may cause performance bottlenecks, which may lead to high network latency and unpredictable cloud service quality degradation [1], [8], [9].

Although, the emergence of 5G technology can alleviate the problem of transmission delay [10]. Unfortunately, users will incur high energy consumption when transferring tasks. Mobile Edge Computing (MEC) is an emerging technology in the 5G era, which can provide nearby services to mobile users [11]. MEC is considered to be a promising method to solve these challenges [12], [13], [14]. Compared with the traditional cloud computing center paradigm, the MEC paradigm provides lower network latency and task transmission energy consumption. MEC servers share the computational burden of mobile/IoT devices through the combination of 5G wireless communication technologies,

- *Shujuan Tian, Chi Chang, Saiqin Long, and Zhetao Li are with the School of Computer Science, Xiangtan University, Xiangtan, Hunan 411105, China, with the Key Laboratory of Hunan Province for Internet of Things and Information Security, Xiangtan University, Xiangtan, Hunan 411105, China, and also with the Hunan International Scientific and Technological Cooperation Base of Intelligent network, Xiangtan University, Xiangtan, Hunan 411105, China. E-mail: {sjtianwork, saiqinlong}@xtu.edu.cn, changchiwork@gmail.com, liztchina@hotmail.com.*
- *Sangyoon Oh is with the School of Information and Computer Engineering, Ajou University, Suwon 443749, South Korea. E-mail: syoh@ajou.ac.kr.*
- *Jun Long is with Big Data and Knowledge Engineering Institute, Central South University, Changsha 410083, China. E-mail: jlong@csu.edu.cn.*

Manuscript received 31 December 2020; revised 30 September 2021; accepted 11 November 2021. Date of publication 16 November 2021; date of current version 6 February 2023.
This work was supported in part by the National Natural Science Foundation of China under Grants 62172349, 62032020, 62076214, and 61902336, in part by the National Key Research and Development Program of China under Grant 2018YFB1003702, in part by the National Natural Science Foundation of Hunan Province under Grants 2019JJ50592 and 2021JJ40544, in part by the Hunan Science and Technology Planning Project under Grant 2019RS3019, and in part by the Hunan Provincial Natural Science Foundation of China for Distinguished Young Scholars under Grant 2018JJ1025.
(Corresponding author: Saiqin Long)
Digital Object Identifier no. 10.1109/TSC.2021.3128603

thereby enhance the rapid response and execution capabilities of mobile/IoT devices to applications [8], [15]. This is especially important for high-traffic applications or timely response systems that require real-time decision-making. Such as, traffic prediction system, medical analysis system, online game system, etc. The MEC has advantages in transmission delay/energy consumption. The disadvantage of MEC is that resources are relatively scarce compared to cloud computing centers. Furthermore, based on the past experience provided by different MEC servers in service quality, charge standard and trust degree etc., different users also have a personalized tendency to choose different MEC servers to execute its diverse tasks in similar regions. That means each user has a preference for selecting MEC servers.

We take the task of an online game (e.g., state synchronization of game players, real-time update of game task operations) for users as an example. These online game requests tend to have large execution data. On the one hand, the user's computing and storage capabilities cannot ensure the real-time completion of the task, and the task needs to be completed by a MEC server or cloud center. Accordingly, the user puts forward the resource requirements on adjacent MEC servers, including communication bandwidth, computing capacity, and data storage amount etc. Obviously, it competes with other users in the region for the resources of the MEC server. On the other hand, there are several optional MEC servers in a certain communication range. Based on the consideration of network traffic service subscriber, the user may prefer the MEC server which is slightly distant to offload its task, rather than considering the traditional geographical distance or the load of the MEC server. Therefore, for different MEC servers, the number of users competing for their resources varies in real time, depending on the preferences of users and when tasks are generated.

As mobile smart devices and IoT device technologies become more mature, the number of mobile/IoT device connections will increase rapidly. It is estimated that the number of connectable devices in 2025 may reach about 390 million [16]. As the number of connected devices increases, the MEC server near the mobile/IoT device also faces the problems of resource shortage, overload, and high latency, etc. However, previous studies mainly focus on the single edge computing environment to perform task offloading, while ignoring the problem of the available resources of the edge server due to many competing users, such as [1], [13], [17].

In this paper, to overcome the above challenges, we study how edge computing and cloud computing work together to provide mobile users with a high-performance computing offloading mechanism. For cloud center, the long-distance task transmission delay is one of the performance that affects the task offloading of mobile users, but the resources are relatively rich. For MEC, the resource of the edge server is relatively shortage than that of the cloud center, it is distributed near mobile users and the transmission path is relatively short. Therefore, when the task of mobile users increases sharply, it will seriously affect the mobile users' quality of services (QoS). Due to the complementary nature of the MEC and cloud center, it is considered that MEC and cloud center cooperate to execute the task requests of mobile users so that mobile users can obtain higher QoS. Therefore we propose a three-tier collaborative service architecture for accessing services, as shown in Fig. 1.

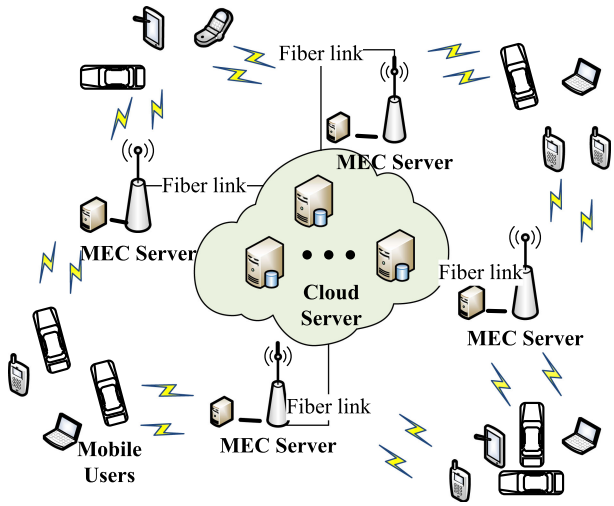


Fig. 1. A three-tier heterogeneous service system of UEs, MECs and the Cloud center.

In addition, with this collaborative service architecture, an efficient offloading strategy is proposed to maximize the number of offloading tasks. Considering the various application characteristics of users, UEs have individual demand for delay and energy consumption. Especially, for personal reasons, each UE also has an individual choosing preference for multiple MECs. So a user's preference factor for MEC is introduced in the offloading strategy. Due to the user's preference, the amount of offloading tasks of some MECs will increase sharply, so that the pressure on these MEC servers will increase. In order to overcome this challenge, we have made dynamic adjustments to some MEC tasks under the premise of considering user preferences as much as possible. Then by comparing complex response ratios to different MECs, all UEs obtain effectively offloading decisions.

1.2 Contributions

The main contributions of this paper are summarized as follows:

- We establish communication and computing models for UEs, MECs, and a single cloud center respectively. Then we calculate the resource consumption function containing response time and energy consumption of tasks and formulate the offloading problem. Taking into account the individual needs of tasks, users can choose the weight value of the delay and energy consumption in the consumption function. Then by comparing complex response ratios to different MECs, all UEs obtain effective offloading decisions. Moreover, we introduce a factor of user preference for MECs, and each UE can adjust its user preference.
- We construct a three-tier heterogeneous service system of multiple UEs, MECs, and a single cloud center. Multiple MECs with different processing capabilities collaborate with the remote cloud center

to jointly form an efficient service architecture. Considering two phases of offloading strategy from UE to MEC and MEC to CC respectively, we propose a response ratio offloading strategy (RROS) algorithm to solve the offloading optimization problem.
- In the RROS algorithm, we propose a task adjustment mechanism based on the response ratio when multiple UEs competing for computing resources of the same MEC at one time. Furthermore, we break the FIFO model of multi-user competition in the initial stage and fully consider the personalized characteristics of multiple UEs. Greedy iteration is used to find the optimal task to be execute first based on the response ratio within a certain interval.
- Experimental simulation results show the superiority of the offloading strategy. Compared with other four offloading methods, the proposed RROS algorithm has good offloading performance with the increasing number of users.

The remainder of this paper is organized as follows: In Section 2, we review related work involving multiple UEs and single MEC server or multiple MEC servers or cloud center. In Section 3, we construct the three-tier offload service architecture and analyze the communication and computing models to get the resource consumption function of UEs. In Section 4, we formulate the task offloading optimization problem for multiple UEs competing for computing resources of multiple MECs and single CC. In Section 5, we develop algorithms to find the optimal offloading decisions for all UEs based on response ratio. In Section 6, we demonstrate the experiment results. Finally, we conclude the paper in Section 7.

2 Related Works

MEC can handle latency-sensitive and computation-intensive applications by deploying computing and storage resources on the edge of a wireless access network, so offloading optimization becomes one of the most important issues in MEC. Many research efforts [18], [19], [20] have been devoted to mobile edge computing or clouding computing. These researches can be grouped into several categories based on the computing network architecture.

Wang *et al.* [17] studied the optimization of computing offloading decision, resource allocation, and content storage in heterogeneous wireless cellular networks of MEC, and solved the problem by applying an alternate direction distributed solution based on multi-threading. You *et al.* [21] studied the optimal resource allocation of a multi-user mobile edge computing offloading system. Under the constraint of computational delay, the goal of optimizing resource allocation was achieved by minimizing the weighted sum of mobile energy consumption. Tran and Pompili [22] studied the joint optimization of task offloading decision, user upline transmission power, and MEC server computing resource allocation in a multi-user and multi-server MEC system networks. In order to solve the resource allocation problem, they used quasi-convex and convex optimization techniques and proposed a heuristic algorithm. Zhang *et al.* [23] proposed an energy-sensing offload strategy that can minimize the weighted sum of delay and energy consumption by jointly optimizing communication resources and computing resources allocation. Dinh *et al.* [24] constructed the scenario where a single user can offload tasks to multiple edges. Guo *et al.* [25] jointed power allocation and subtask allocation problems with convex optimization methods. Mao *et al.* [26] investigated the tradeoff between two critical but conflicting objectives (the power consumption of mobile devices and the execution delay of computation tasks) in multi-user MEC systems, then resolved a stochastic optimization problem. Yang *et al.* [27] presented a multi-user and multi-edge computing partition problem, and solved computing and resource allocation optimization problems by maximizing the average throughput of users.

All the above studies [17], [21], [22], [23], [24], [25], [26], [27] focused on a two-tier computing network consisting of multiple UEs and a single MEC server or multiple MEC servers.

There are also a few articles that study the offloading problem in the three-tier computing network composed of mobile devices, edge computing nodes, and cloud center. In [28], by the joint optimization of offloading decision and allocation of communication and computing resources, the goal of minimizing the total cost of the system was achieved. And in the case of ignoring resource allocation, a centralized heuristic algorithm for offloading decision was proposed in [29] to enhance and optimize the computing power of mobile devices. Liu *et al.* [30] established a multi-objective optimization problem by queuing theory to optimize indicators such as delay, energy consumption, payment, and obtained the optimal offloading probability and transmission power of the service. Chen *et al.* [31], [32] considered the three-tier service offloading framework of the mobile device. They proposed an efficient solution named SHAREAPP by semidefinite relaxation and a randomization mapping method to optimize the users offloading decision. However, with one computing access point, they neglected the cooperative optimization of multiple MECs.

It is a pity that current approaches do not take the individual needs of users and the communication consumption of computing resources into account. Undoubtedly they affect resource allocation and offloading decisions. Furthermore, the tasks generated from multiple users change in real time. To address this weakness, we consider not only the individual needs of users for delay and energy, but also user preference for different MEC servers. Then we propose a method to deal with the real-time of tasks offloading problem.

3 System Model

In this paper, we study the issue of efficient computation offloading strategy for multiple UEs to offload computationally intensive tasks. At first, we construct a heterogeneous network architecture with multiple UEs, multiple MECs servers and a single Cloud Center, as illustrated in Fig. 1, where there are $\mathcal{N} = \{1, 2, \ldots, N\}$ mobile user services and $\mathcal{M} = \{1, 2 \ldots, M\}$ MECs, i.e., $UE_1, UE_2, \ldots, UE_N$, MEC_1, $MEC_2, \ldots, MEC_M$, and CC. Vertically, multiple UEs with personalized requirements compete for computing resources on multiple MECs and multiple MECs compete for

cloud center computing resources based on their processing capabilities and computing requirements of corresponding UEs. Horizontally, multiple UEs are selfish and compete with other UEs in geographical proximity. Multiple MECs form a flat structure to serve nearby UEs. Any mobile edge MEC_m is a single server system with classes of tasks from different UEs. To be simple, supposing all MEC servers are deployed on a base station or an AP. As the access network infrastructure, base station or AP is wired connected to the cloud center.

Any mobile user UE_n has a computationally intensive task $\tau_n = \langle b_n, w_n, d_n \rangle$, where b_n is the amount of data to be computing, w_n is the execution requirement of the computation task generated on UE_n, and d_n is the quality requirements, which is set to be execute time limit in this paper. The values of b_n and w_n depend on the nature of the calculation task and can be obtained by offline measurement [33]. In order to be more realistic, any mobile user UE_n has different computing power f_n^l (i.e., CPU operations per second).

Any mobile edge MEC_m is a single server system with classes of tasks from different UEs and we establish a M/M/1 queueing model for the MEC_m. f_n^m denotes the computing power of MEC_m and $f_1^m = f_2^m = , \ldots, = f_N^m$. If there are more than one task, the tasks queue in a buffer. The only Cloud Center(CC) is located away from mobile UEs and MECs. It has powerful computing ability with computing power f_n^c to task τ_n, and $f_1^c = f_2^c = , \ldots, = f_N^c$.

With different computing abilities to deal with different kinds of tasks, a three-tier heterogeneous and distrubuted offloading network is constructed. To efficiently execute compute-intensive or delay-sensitive applications, there are two phases of computing offloading strategies: first phase is to make a decision whether offload task from UE_n to MEC_m or execute locally at UE_n; second phase is to make a decision whether offload task from MEC to CC or wait in a queue to execute at MEC. We denote a_n as the first phase offloading decision and x_n as the second phase offloading decision of task τ_n from UE_n. When $a_n = 1$ means that the computing task τ_n at this time is offloaded to MEC for execution. Otherwise, $a_n = 0$ means that τ_n is executed on the local CPU of UE_n. When $x_n = 0$, the task τ_n will be computed on MEC and $x_n = 1$ means the task is continued to offload to CC for execution. In this way, for all UEs, we can get two decision vectors $\boldsymbol{a} = (a_1, \ldots, a_n, \ldots, a_N)$ and $\boldsymbol{x} = (x_1, \ldots, x_n, \ldots, x_N)$. For reader's convenience, we provide Table 1, which gives a summary of major notations and definitions in the paper.

3.1 Communication Model

3.1.1 Communication Between UE and MEC

In the communication model, bandwidth B_m allocation corresponds to a circular scheduling rule. It also shows that these devices associated with MEC share resources equally. According Shannon's formula, we denote the uplink transmission rate $r_{n,m}$ between UE_n and MEC_m as

$$r_{n,m} = B_m \log\left(1 + \frac{p_{tn} g_{n,m}}{N_0}\right) \tag{1}$$

As Eq. (1) shows, p_{tn} is the transmission power of device UE_n. $g_{n,m}$ is the channel gain between the device UE_n and the MEC_m. N_0 is the power of the additive white Gaussian noise. Correspondingly, the transmission time of task τ_n from UE_n to MEC_m is

$$t_{n,m}^{tr} = \frac{b_n}{r_{n,m}}. \tag{2}$$

Generally, the download rate of data is much higher than the upload rate. Assume that the transmission rate from UE_n to MEC_m is ξ times the transmission rate from MEC_m to UE_n, then the computing result return time is

$$t_{n,m}^{down} = \frac{D_n}{\xi r_{n,m}}, \tag{3}$$

where D_n is the amount of result data.

3.1.2 Communication Between MEC and CC

As stated before, the MEC servers and the CC are wired connected through the backbone network. Supposing R_{ec} is the transmit data rate between any MEC server and CC. For convenience, assuming R_{ec} does not change during the execution process and the upstream and downstream transmit rates are the same. Therefore, the transmit time of task τ_n from MEC_m to CC is

$$t_{n,c}^{tr} = \frac{b_n}{R_{ec}}. \tag{4}$$

Correspondingly, the return time $t_{n,c}^{down}$ of CC which return the result to MEC_m is

$$t_{n,c}^{down} = \frac{D_n}{R_{ec}}. \tag{5}$$

3.2 Computing Model

In most existing computation offload strategies, the optimized performance index point is either to reduce the calculation delay, or to reduce the energy consumption, or to comprehensively consider the delay and energy consumption together. Considering that each mobile user is an individual with different needs, this paper proposes a personalized computing task offloading model to meet the personalized needs of different mobile users.

3.2.1 Local Computing Model

When a mobile device with certain processing capabilities and sufficient energy, it can deal computing task locally. In this way, the local response time of task τ_n only depends on local computing ability of CPU f_n^l.

$$t_{n,l} = \frac{b_n w_n}{f_n^l} \tag{6}$$

And the energy consumed by τ_n of UE_n is

$$e_{n,l} = p_n^l t_{n,l}, \tag{7}$$

where p_n^l is the local CPU execution power of UE_n.

Then we can build a resources consumption function for task τ_n of UE_n as

$$\Psi_{n,l} = \beta_n^t t_{n,l} + \beta_n^e e_{n,l}. \tag{8}$$

TABLE 1
Summary of Major Notations and Definitions

Notation	Definition
$\mathcal{N}$	the set of mobile users (UE_s).
$\mathcal{M}$	the set of MEC_s.
τ_n	a computationally intensive task of mobile user UE_n.
b_n	the amount of UE_n's data to be computing.
w_n	the execution requirement of the computation task generated on UE_n.
d_n	the quality requirements of UE_n, it is set to be the execute time limit
f_n^L	the computing power of mobile user UE_n (CPU operations per second).
$t_{n,l}$	the local response time of task τ_n only depends on local computing ability of CPU f_n^L.
p_n^L	the local CPU execution power of UE_n (CPU operations per second).
$e_{n,l}$	the energy consumed by UE_n for τ_n.
β_n^t, β_n^e	represent the weights of response time and energy of UE_n respectively.
$\Psi_{n,l}$	a local overall load model for task τ_n of UE_n.
$r_{n,m}$	the offloading rate of UE_n to MEC_m.
$t_{n,m}^{tr}$	the transmission time of task τ_n from UE_n to MEC_m.
$e_{n,m}^{up}$	the energy consumption of UE_n transmiting the task to MEC_m.
f_n^m	the computing power of the virtual machine allocated by MEC_m server to UE_n.
$t_{n,m}^{wait}$	the waiting time of UE_n.
$\eta_{n,m}$	the response ratio of UE_n.
$t_{n,m}^{exe}$	the computing time at MEC_m.
D_n	the amount of result data.
$t_{n,m}^{down}$	the computing result return time from MEC_m.
p_{rn}, p_{tn}	the receive (transmission) power of UE_n.
B_m	the communication bandwidth allocated to a task of MEC_m.
$e_{n,m}^{down}$	the receive energy consumption of UE_n.
$t_{n,m}$	the response time of task τ_n at this MEC Computing Model.
$e_{n,m}$	the energy consumption of receiving the computing results.
$\Psi_{n,s}$	the overall offloading load model for task τ_n to MEC.
L_n^m	the difference between local metrics $\Psi_{n,l}$ and offloading metrics $\Psi_{n,s}$.
$t_{n,m}^{me}$	the time consumption of task τ_n in MEC_m.
R_{ec}	represent the communication rate of MEC_m offloading task to CC in round-trip.
f_n^c	the computing capability of CC for offloading task.
$t_{n,c}^{up}$	the transmission time from MEC_m to CC.
$t_{n,c}^{down}$	the transmission time from CC to MEC_m.
$t_{n,c}^{exe}$	the execute time on CC.
$t_{n,c}^{cl}$	the time consumption of the second offloading process on CC.
$t_{n,c}$	the totol response time of task in CC Computing Model.
a_n	the first phase offloading decision of task τ_n from UE_n.
x_n	the second phase offloading decision of task τ_n from UE_n.
L_n^m	the difference of overall load between local and offloading computing model.
δ_m	the bias value for a certain MEC.

where β_n^t and β_n^e respectively represent the weights of response time and energy consumption of UE_n, and they follow as

$$\begin{cases} \beta_n^t + \beta_n^e = 1, \\ 0 \le \beta_n^t \le 1, \\ 0 \le \beta_n^e \le 1. \end{cases} \tag{9}$$

When β_n^t is larger, it means that the mobile device is more concerned with the response time and it is more sensitive to the delay; while β_n^e is larger, it means that the mobile device is concerned with energy consumption. In this way, different mobile users with different tasks can appropriately select the weighting coefficient according to their specific situation at one time.

3.2.2 MEC Computing Model

When UE_n decides to offload its computing task to MEC for execution, the mobile device first needs to transmit the task to MEC through a wireless channel, then MEC replaces the mobile device to execute the computing task. At this time, the computing task needs to go through three steps to be completed: step1. task offloading to MEC which belongs to the first offloading strategy; step2. task computing or queueing at MEC or further offload to CC which belongs to the second offload strategy; step3. computing result return.

During step1, UE_n requires additional delay and energy consumption to transmit the task to MEC. The offloading delay can be obtained as Eq. (2): $t_{n,m}^{tr} = \frac{b_n}{r_{n,m}}$.

And in step2, the computing time at MEC is

$$t_{n,m}^{exe} = \frac{b_n w_n}{f_n^m}, \tag{10}$$

where f_n^m is the computing power of the virtual machine allocated by MEC_m server to UE_n.

As we described earlier, any MEC server serves up to C UEs at a time. Therefore, when the computing resources of MEC_m is insufficient, some tasks are waiting in the queue of MEC_m or be offload to CC. For any task τ_n, its waiting

time on MEC_m is $t_{n,m}^{wait}$. Obviously, if the computing resources of MEC_m is insufficient, $t_{n,m}^{wait} \geq 0$, otherwise $t_{n,m}^{wait} = 0$. And then we introduce the response ratio $\eta_{n,m}$ of UE_n which is completion time divided by service time in MEC_m. Then it can be represented as follows

$$\eta_{n,m} = \frac{t_{n,m}^{wait} + t_{n,m}^{exe}}{t_{n,m}^{exe}} = 1 + \frac{t_{n,m}^{wait}}{t_{n,m}^{exe}} \tag{11}$$

And it can be denoted by C as the number of UEs offloaded to MEC_m, and $\phi_m = \{1, 2, 3 \dots C\}$ as the task execution sequence on MEC_m, which satisfies $\eta_{1,m} > \eta_{2,m} > \eta_{3,m} > \dots > \eta_{C,m}$ i.e., So the waiting time $t_{n,m}^{wait}$ of UE_n is

$$t_{n,m}^{wait} = \sum_{i=1}^{n-1} t_{i,m}^{exe} \tag{12}$$

Therefore, the time consumption of task τ_n in MEC_m is

$$t_{n,m}^{me} = t_{n,m}^{wait} + t_{n,m}^{exe}. \tag{13}$$

The computing result return time can be obtained as Eq. (3): $t_{n,m}^{down} = \frac{D_n}{\xi r_{n,m}}$. And the response time of task τ_n at this MEC Computing Model is

$$t_{n,m} = t_{n,m}^{tr} + t_{n,m}^{me} + t_{n,m}^{down}. \tag{14}$$

No matter the task is offload to MEC or CC for execution, the energy consumption of UE_n to transmit the task is

$$e_{n,m}^{up} = p_{tn} t_{n,m}^{tr} + L_n = \frac{p_{tn} b_n}{r_{n,m}} + L_n, \tag{15}$$

where L_n is the inherent energy consumption of the mobile device transmitter. This extra energy consumption is a common phenomenon in all mobile terminals [34].

And the receive energy consumption of UE_n is

$$e_{n,m}^{down} = \frac{p_{rn} D_n}{\xi r_{n,m}} \tag{16}$$

where p_{rn} is the receiver power of UE_n.

Then, for UE_n, the total energy consumption on MEC computing model is

$$e_{n,m} = e_{n,m}^{up} + e_{n,m}^{down} \tag{17}$$

Therefore, the resources consumption function of MEC computing model for task τ_n is

$$\Psi_{n,m} = \beta_n^t t_{n,m} + \beta_n^e e_{n,m}. \tag{18}$$

And the difference L_n^m of the resources consumption function between local computing and offloading computing model is

$$L_n^m = \Psi_{n,l} - \delta_m * \Psi_{n,m}. \tag{19}$$

where δ_m is the bias value for a certain MEC. If the user prefers to uninstall a certain MEC, δ_m can take a certain minimum constant. If there is no specific bias, the value of δ_m defaults to 1.

So when $\delta_m = 1$, the value of a_n is

$$a_n = \begin{cases} 1 & if \quad \Psi_{n,l} > \Psi_{n,m} \\ 0 & otherwith. \end{cases} \tag{20}$$

3.2.3 CC Computing Model

When the computing resources of MEC_m is insufficient, the second offloading strategy decides whether continue to offload and execute the task on the cloud center or wait in MEC_m in queue. The task is further offload to CC When $x_n = 1$. The time consumption of the second offloading process contains three parts, one is the transmission time as Eq. (4): $t_{n,c}^{tr} = \frac{b_n}{R_{ec}}$ from MEC_m to CC, one is the execute time on CC, and the last part is the time as Eq. (5): $t_{n,c}^{down} = \frac{D_n}{R_{ec}}$ that CC return the result to MEC_m. the execute time $t_{n,c}^{exe}$ on CC is

$$t_{n,c}^{exe} = \frac{b_n w_n}{f_n^c}. \tag{21}$$

where f_n^c is the computing capability of CC for task τ_n. The time consumption of the second offloading process on CC is

$$t_{n,c}^{cl} = t_{n,c}^{up} + t_{n,c}^{exe} + t_{n,c}^{down} \tag{22}$$

Therefore, the total response time of task in CC Computing Model is

$$t_{n,c} = t_{n,m}^{tr} + t_{n,c}^{cl} + t_{n,m}^{down}. \tag{23}$$

For UE_n, the total energy consumption in CC Computing Model is the same with that in MEC Computing Model, as Eq. (17) shows. Then we analysis the key factor $t_{n,m}^{wait}$ which impact the second phase offloading strategy. As we described before, if the waiting time of task τ_n or computing time in MEC_m is greater than the extra time consumption to offload to CC for execution, UE_n will make the second phase strategy to offload task to CC. Thus x_n can be calculate by

$$x_n = \begin{cases} 1 & if \quad t_{n,m}^{me} \geq t_{n,c}^{cl}, \\ 0 & otherwith. \end{cases} \tag{24}$$

Therefore, the resources consumption function of CC computing model for task τ_n is

$$\Psi_{n,c} = \beta_n^t t_{n,c} + \beta_n^e e_{n,m}. \tag{25}$$

3.2.4 Server Computing

To conclude, the resources consumption function for task τ_n to local or offload to MEC_m or CC is as follow.

$$\Psi_{n,s} = x_n \cdot \Psi_{n,m} + (1 - x_n) \cdot \Psi_{n,c}; \tag{26}$$

$$\Psi_n = a_n \cdot \Psi_{n,s} + (1 - a_n) \cdot \Psi_{n,l}; \tag{27}$$

4 Offloading Strategy

4.1 Problem Formulation

As mentioned earlier, in order to deal with its tasks efficiently, each mobile user can offload the tasks to MEC servers for execution. From the perspective of MEC, providing services to more users has higher user satisfaction. From the perspective of UEs, each UE is selfish and competitive to offload its tasks as much as possible within the prescribed time limit. Only obtaining greater benefits than that of local execution by offloading tasks to MECs or CC for execution, will UEs be motivated to perform tasks offloading. Moreover, we must note that when a UE makes a decision of computation offloading, the UE should be aware of the fact that MECs are serving other UEs. So offloading strategy of any UE affects the workload of MECs, which in turn affects the formulation of other UEs' offloading strategies.

Therefore, the goal of this paper is to maximize the number of offloading tasks while meeting their respective time constraints. Supposing $\mathcal{S}_m \subseteq \mathcal{N}(m \in M)$ indicate a group of tasks from different mobile users to be offloaded to MEC_m for execution. Then, we formulate the problem Q1 as

$$\max \sum_{m \in M} |\mathcal{S}_m| \tag{28}$$

s.t.

$$\mathcal{S}_m \cap \mathcal{S}_{-m} = \varnothing, \forall \mathcal{S}_m, \mathcal{S}_{-m} \subseteq \mathcal{N} \tag{29}$$

$$\Psi_{n,l} \geq \Psi_{n,m}, \forall m \in M, \forall \tau_n \in \mathcal{S}_m \tag{30}$$

$$\Gamma_n\left(\{\mathcal{S}_m\}_{m \in M}\right) \leq d_n, \forall \tau_n \in \mathcal{S}_m \tag{31}$$

where $|\mathcal{S}_m|$ is the amount of tasks from different mobile users.

Constraint (29) denotes that one task is offloaded to one MEC for execution. Constraint (30) means that any tasks will only be offload to a MEC or CC when the offload is more profitable than the local one. Constraint (31) ensures that each task τ_n meets its own time limit of d_n.

In order to derive the two phases of offload strategies, we can compare the total offloading load in the local model to that in the offloading model. Then another objective of this paper is to maximize the number of UEs that gains from computing offloading in a multi-user scenario as

$$\max \sum_{n \in N} I^{n,m}_{\{a_n \geq 0\}} \tag{32}$$

s.t.

$$\Psi_{n,l} \geq \Psi_{n,m}, \forall a_n \geq 0, \forall n \in N \tag{33}$$

$$\Gamma_n\left(\{\mathcal{S}_m\}_{m \in M}\right) \leq d_n, \forall \tau_n \in \mathcal{S}_m \tag{34}$$

$$\sum_{m=1}^{M} I^{n,m}_{\{a_n > 0\}} \leq 1, \forall n \in N \tag{35}$$

$$\sum_{n=1}^{N} I^{n,m}_{\{a_n > 0\}} \leq 1, \forall m \in M \tag{36}$$

$$a_n \in \{0, 1\}, \forall n \in N \tag{37}$$

where $\Gamma_n\left(\{\mathcal{S}_m\}_{m \in M}\right)$ denotes the response time of τ_n in the strategy of **a**. and $I^{n,m}_{\{a_n \geq 0\}}$ is an indicator function, defined as follows

$$I^{n,m}_{a_n \geq 0} = \begin{cases} 1 & a_n = 1 \\ 0 & a_n = 0 \end{cases} \tag{38}$$

At this time, the task offloading strategy problem can be described as the maximum number of UEs which offload tasks at one time in a multi-user scenario. For the maximum packing problem of multiple boxes, the NP difficulty of this problem can be derived.

Proof To prove this problem, we first introduce the biggest packing problem for multiple boxes [35]. Given N objects, the volume of each object is v_i, $i \in \{1, 2, \ldots, N\}$, and there are M boxes and the capacity of each box is V_c. The goal of the problem is to pack objects into boxes so that the maximum number of objects can be packed into boxes. The problem can be described as follows.

$$\max \sum_{i=1}^{N} \sum_{j=1}^{M} x_{i,j} \tag{39}$$

s.t.

$$\sum_{i=1}^{N} v_i x_{i,j} \leq V_c, j \in \{1, 2, \ldots, M\} \tag{40}$$

$$\sum_{i=1}^{N} x_{i,j} \leq 1, i \in \{1, 2, \ldots, N\} \tag{41}$$

$$x_{i,j} \in \{0, 1\}, i \in \{1, 2, \ldots, N\}, j \in \{1, 2, \ldots, M\} \tag{42}$$

Now we compare UEs to the objects in the biggest packing problem, MEC is compared to the box of the biggest packing problem. The volume v_i of the object can be described as $b_n w_n$, the capacity V_c of the box can be expressed as the number of tasks that MEC can process at a time. In this way, we turn the problem into the biggest packing problem for multiple objects, because the packing problem is NP-hard, our problem is also NP-hard. The proof has been completed. □

4.2 Optimization Solution

Based on the above analysis, we can see that there are two main steps to get a complete offloading scheme: 1) each MEC collects all tasks competing for the MECs resources; 2) each MEC sorts the tasks. To solve the above problems, we propose a response ratio offloading scheduling algorithm (RROS), and give the optimization scheme as following.

4.2.1 Task Set Initialization

Initialize tasks collection as follows:

Step 1: Calculate the maximum difference L^m_n of the resources consumption function between local computing consumption function $\Psi_{n,l}$ and offloading computing consumption function $\Psi_{n,m}$ that set the waiting time $t^{wait}_{n,m} \leftarrow 0$ to select which edge the user is offloaded to. Then can get an initialization list for each edge $K_m \leftarrow K_m \cup n(m \in \mathcal{M}, n \in \mathcal{N})$;

Step 2: For each MEC, arrange all tasks in an initial sequence.

4.2.2 Dynamic Adjustment of Task Set

Dynamic adjustment of task set means that the tasks from MEC_{m_h} (transfer-out MEC) are transferred to another MEC_{m_l} (transfer-in MEC). We adopt the following ways to find the transfer-out and transfer-in MECs.

$$\Psi_m = \sum_{n \in K_m} \Psi_n, m \in \mathcal{M} \tag{43}$$

Ψ_m denotes the sum of resources consumption for task τ_n, $n \in K_m$. $\mathcal{W} = \{\Psi_1, \Psi_2, \ldots, \Psi_M\}$. $\mathcal{L}_m$ stores the tasks which are offloaded to MEC or cloud, $\mathcal{C}_m$ stores the tasks which are executed locally.

Obviously, when a users task has a large resource consumption function on one MEC, the user will naturally seek another MEC with a smaller resource consumption function to offload. Therefore, we optimize the offloading decisions globally by adjusting the tasks on the MECs with high resource consumption to the MECs with small resource consumption. Based on this, we update the transfer-out and transfer-in MECs in the following steps.

Step 1: Calculate $\Psi_m, \forall m \in \mathcal{M}$;

Step 2: Sort $\Psi_m \in \mathcal{W}$ in a descending order;

Step 3: Obtain the transfer-out MEC_{m_h}

$$m_h = \arg\max_{m \in \mathcal{M}} \Psi_m \in \mathcal{W} \tag{44}$$

Step 4: Obtain the transfer-in MEC_{m_l}

$$m_l = \arg\min_{m \in \mathcal{M}} \Psi_m \in \mathcal{W} \tag{45}$$

Step 4: Change the task τ_n randomly, $n \in \mathcal{C}_m$, from MEC_{m_h} to MEC_{m_l}.

If $\mathcal{C}_m = \phi$, we change the task τ_n randomly, $n \in \mathcal{L}_m$, from MEC_{m_h} to MEC_{m_l}. And then we obtain a new task assignment decision.

4.2.3 Sort Tasks in MEC

Define

$$\overline{T}_m = \sum_{n=1}^{|S_m|} \frac{b_n w_n}{f_n^m} + \max_{n \in |S_m|} t_{n,m}^{tr} \tag{46}$$

$\overline{T}_m$ is time delay of MEC_m ($m \in \mathcal{M}$) in the worst offloading environment, all reasonable offloading tasks will be complete by the time $\overline{T}_m$. Thatmeans all tasks are be processed during the time interval $[\min_{n \in |S_m|, m \in M} t_{n,m}^{tr}, \overline{T}_m]$. Then we split the interval into many equal time length as Δ.

$$H = \left\lceil \frac{\overline{T}_m - \min_{n \in |S_m|, m \in M} t_{n,m}^{tr}}{\Delta} \right\rceil \tag{47}$$

where $\lceil Z \rceil$ denotes the ceiling function, it maps Z to the least integer greater than or equal to Z. H is the number of time slots. We denote a set of time slots as $\mathcal{H} = \{1, \ldots, H\}$.

Therefore, for any task τ_n, the execution start time in MEC will be in the time interval $[t_{n,m}^{tr} + t_{n,m}^{wait}, d_n - t_{n,m}^{exe}]$. For convenience, we denote $\underline{h}_n^m$ as the time slot which contains the time $\lceil \frac{t_{n,m}^{tr} + t_{n,m}^{wait}}{\Delta} \rceil$, and $\bar{h}_n^m$ as the time slot which contains the time $\lceil \frac{d_n - t_{n,m}^{exe}}{\Delta} \rceil$. That means τ_n starts at any time in the interval of $[\underline{h}_n^m, \bar{h}_n^m]$. Let $\Upsilon_{n,m} (n \in N, m \in M)$ be the set of available starting time slots for the task τ_n on MEC_m, then

$$\Upsilon_{n,m} = \begin{cases} \varnothing, & if \quad \bar{h}_n^m < \underline{h}_n^m; \\ [\underline{h}_n^m, \bar{h}_n^m], & otherwise. \end{cases} \tag{48}$$

And the number of slots transmitting and executing τ_n as well as waiting on MEC_m is

$$s_{n,m}^{tr} = \left\lceil \frac{t_{n,m}^{tr}}{\Delta} \right\rceil \tag{49}$$

$$s_{n,m}^{exe} = \left\lceil \frac{t_{n,m}^{exe}}{\Delta} \right\rceil \tag{50}$$

$$s_{n,m}^{wait} = \left\lceil \frac{t_{n,m}^{wait}}{\Delta} \right\rceil \tag{51}$$

Let $y_{n,m,s}$ be the indicator whether to start execution task τ_n on MEC_m and $x_{n,m,s}$ be the indicator whether to start execution task τ_n on CC in the sth($s \in \mathbf{H}$) slot. Obviously, in the same slot, when $y_{n,m,s} = 0$, $x_{n,m,s} = 0$. When $y_{n,m,s} = 1$, the value of $x_{n,m,s}$ depends on the comparison of $t_{n,m}^{me}$ and $t_{n,c}^{cl}$. As Eq. (24) shows, $s_{n,c}^{cl} = \left\lceil \frac{t_{n,c}^{cl}}{\Delta} \right\rceil$, and if $x_{n,m,s} = 1$, $y_{n,m,s} = 0$.

Then the problem can be transformed as the following:

$$\max \sum_{n \in N, m \in M, s \in \Upsilon_{n,m}} y_{n,m,s} + x_{n,m,s} \tag{52}$$

s.t.

$$\sum_{m=1, s \in \mathcal{H}}^{M} y_{n,m,s} \leq 1, \forall n \in N \tag{53}$$

$$\sum_{n=1, s \in \mathcal{H}}^{|S_m|} y_{n,m,s} \leq 1, \forall m \in M \tag{54}$$

$$y_{n,m,s} \in \{0,1\}, \forall n \in N, \forall m \in M, \forall s \in \mathcal{H} \tag{55}$$

$$\sum_{n \in N, s \in [\{1, h - \eta_{n,m}\}, h]} y_{n,m,s} \leq 1, \forall m \in M, \forall h \in \mathcal{H} \tag{56}$$

$$\sum_{m=1, s \in \mathcal{H}}^{M} y_{n,m,s} x_{n,m,s} = 0, x_{n,m,s} \in \{0,1\}, \forall n \in N \tag{57}$$

$$y_{n,m,s} = 0, \forall n \in N, \forall m \in M, \Psi_{n,l} \leq \Psi_{n,m}, \Upsilon_{n,m} = \varnothing. \tag{58}$$

Constraint (53) and (55) denote that one task is offloaded to one MEC for execution, constraint (54)and (55) enforce that each MEC can serve at most one tasks at a time. Constraint (56) means that at most one task is processed on any MEC at any time. Constraint (57) denotes that there is only one offloading decision for a task. The last constraint

ensures that each task won't be processed before its arrival time.

Therefore, two phase offloading strategies of any task τ_n can be obtained by

$$x_n = \sum_{m=1}^{M} \sum_{n=1, s \in \mathcal{H}}^{|S_m|} x_{n,m,s}. \tag{59}$$

$$a_n = \sum_{m=1}^{M} \sum_{n=1, s \in \mathcal{H}}^{|S_m|} y_{n,m,s} + x_n \tag{60}$$

From the above analysis, the task assignment and adjustment process can be summarized as follows.

1) All MECs are divided into transfer-out and transfer-in MECs according to the distribution of tasks competing for their resources.

2) The tasks in transfer-out and transfer-in MECs are dynamically adjusted.

3) All tasks in each MEC are sorted based on the response ratio.

5 The Response Ratio Offloading Scheduling Algorithm

In order to solve this problem, we propose an offloading decision and task scheduling algorithm. The offloading strategy mainly consists of three parts:

1) MEC selecting

- First, we know that different UEs with different tasks can appropriately select the weighted coefficient according to their specific situation. In Algorithm 1, we calculate the local overall load $\Psi_{n,l}$ as Eq. (8) and the MEC offloading load model $\Psi_{n,m}$ as Eq. (18) with $t_{n,m}^{wait} \leftarrow 0$ for task τ_n of UE_n. And according to user preference factors, calculate the difference L_n^m.

Algorithm 1. Find K_m

Input: b_n, w_n, d_n, f_n^L, p_n^L, β_n^t, β_n^e, g_n, B, N_0, f_n^m, $p_{r,n}$, p_{tn}, L_n, D_n, R_{ac}, f_n^c, R_{ec}.
Output: K_m.
1: Initialize $K_m \leftarrow \phi$.
2: **for** $n \in \mathcal{N}$ **do**
3: **for** $m \in \mathcal{M}$ **do**
4: Calculate $\Psi_{n,l}$ as Eq. (8); And calculate $\Psi_{n,m}$ as Eq. (18) with $t_{n,m}^{wait} \leftarrow 0$; Then calculate L_n^m as Eq. (19);
5: **if** $L_n^m > 0$ **then**
6: $m \leftarrow \arg\max_{m \in \mathcal{M}} L_n^m$;
7: **end if**
8: **end for**
9: $K_m \leftarrow K_m \cup \{\mathrm{n}\}$;
10: **end for**
11: **return** K_m.

- Then get the max difference L_n^m between $\Psi_{n,l}$ and $\Psi_{n,m}$ to select which MEC the user is offloaded to. Then create a preference list $K_m \leftarrow K_m \cup n (m \in \mathcal{M}, n \in \mathcal{N})$.

2) Tasks scheduling

Algorithm 2. Response Ratio Offloading Scheduling (RROS) Algorithm

Input: K_m.
Output: $y_{n,m,s}$, $x_{n,m,s}$.
1: Initialize $q_0 \leftarrow 0, \Psi_m \leftarrow 0, \mathcal{W} \leftarrow \phi$, $\mathcal{L}_m \leftarrow \phi$, $\mathcal{C}_m \leftarrow \phi$;
2: **for** $m \in \mathcal{M}$ **do**
3: Set $\mathcal{Q}_m \leftarrow \phi$;
4: **for** $n \in K_m$ **do**
5: **if** $s_{n,m}^{tr} < \bar{h}_n^m$ **then**
6: Set $\mathcal{Q}_m \leftarrow \mathcal{Q}_m \cup \{n\}$;
7: **end if**
8: **end for**
9: $q_0 \leftarrow$ length of $\mathcal{Q}_m$;
10: **if** $q_0 \leq 1$ **then**
11: **if** $q_0 = 1$ **then**
12: $y_{n,m,s} = 1$;
13: **else**
14: $y_{n,m,s} = 0$;
15: **end if**
16: **else**
17: Obtain starter UE by using Algorithm 3;
18: Obtain $y_{n,m,s}$ and $x_{n,m,s}$ by using Algorithm 4;
19: **end if**
20: Calculate Ψ_m as Eq. (43); $\mathcal{W} \leftarrow \mathcal{W} \cup \Psi_m$; $\mathcal{L}_m \leftarrow K_m \cap K_m \setminus \{\mathcal{C}_m\}$
21: **end for**

- By Algorithm 1, we get the preference list K_m to the MECs. And in Algorithm 4, in MEC_m we set a list $\mathcal{Q}_m$ which can be used to store the UEs that meet $s_{n,m}^{tr} < \bar{h}_n^m$(Steps 4-7). Then we find the optimal offload starter UE $\mathcal{V}_m[0]$ based on response ratio within a certain interval $\mathcal{G}$ by using Algorithm 3, and obtain the offloading decision $y_{n,m,s}$ and $x_{n,m,s}$ by using Algorithm 4.
- In Algorithm 3, we get the min arrival time $s_{i,m}^{tr}$ of UE_n in MEC_m. When the successively arrival time interval of tasks is within a certain interval $\mathcal{G}$, their response ratio $\eta_{n,m}$ should be required(Steps 3-7). Then calculate the max response ratio and get label value n of UE $n \leftarrow \arg\max_{n \in \mathcal{N}} \eta_{n,m}$, finally we obtain the optimal offload starter UE $\mathcal{V}_m[0]$(Steps 9-18).
- When get the starter UE $\mathcal{V}_m[0]$ from Algorithm 3, we use $\mathcal{U}_m$ to store the response ratio $\eta_{n,m}$ of the remaining tasks which meet the conditions of the $\mathcal{V}_m^{tr}[0] + \mathcal{V}_m^{exe}[0] \geq s_{n,m}^{tr}$ in MEC_m(Steps 1-6). And then judge whether $\mathcal{U}_m$ is empty or not. If $\mathcal{U}_m$ is empty, repeat the above procedures by using Algorithms 3 and 4. If not, we sort the response ratio $\eta_{n,m} \in \mathcal{U}_m$ in a descending order and get UEs by $n \leftarrow \arg_{n \in \mathcal{N}} \eta_{n,m}$(Steps 6-10).
- After getting sorted results, calculate the respective waiting time and use Eqs. (48) and (24) to judge whether it should be further transferred to the cloud, queued or put in the local calculation(Steps 11-22). And then judge whether $\mathcal{V}_m$ is empty. If $\mathcal{V}_m$ is empty, repeat the above procedures by using Algorithms 3 and 4. If not, repeat the above procedures by using

Algorithm 3. Find Starter UE $\mathcal{V}_m[0]$

Input: $\mathcal{Q}_m$.
Output: $\mathcal{V}_m[0]$.
Initialize $\mathcal{N}_U \leftarrow \phi$; $\mathcal{V}_m \leftarrow \phi$;
  $i \leftarrow \arg\min_{i \in \mathcal{N}} s_{i,m}^{tr}$;
  for $n \in \mathcal{Q}_m$ **do**
    if $s_{n,m}^{tr} - s_{i,m}^{tr} < \mathcal{G}$ **then**
      $\mathcal{N}_U \leftarrow \{n\}$;
    end if
  end for
  $q_1 \leftarrow$ length of $\mathcal{N}_U$;
  if $q_1 = 1$ **then**
    Set $\mathcal{V}_m[0] \leftarrow i$; $y_{i,m,s} = 1$;
  else
    for $j \in \mathcal{N}_U$ **do**
      for $n \in \mathcal{N}_U \setminus \{j\}$ **do**
        $n \leftarrow \arg\max_{n \in \mathcal{N}} \eta_{n,m}$;
      end for
    end for
    $y_{n,m,s} = 1$; $\mathcal{V}_m[0] \leftarrow n$;
end if

Algorithms 4(Steps 24-28). It does not end until $\mathcal{Q}_m = \phi$.

3) Offloading strategy adjustment

Algorithm 4. Find Offloading Scheduling $y_{n,m,s}$ and $x_{n,m,s}$

Input: $\mathcal{Q}_m$, $\mathcal{V}_m[0]$;
Output: $y_{n,m,s}$, $x_{n,m,s}$.
Initialize $\mathcal{U}_m \leftarrow \phi$;
for $n \in \mathcal{Q}_m \setminus \{\mathcal{V}_m[0]\}$ **do**
  if $\mathcal{V}_m^{tr}[0] + \mathcal{V}_m^{exe}[0] \geq s_{n,m}^{tr}$ **then**
    Calculate $\eta_{n,m}$ as Eq. (11); Set $\mathcal{U}_m \leftarrow \mathcal{U}_m \cup \{\eta_{n,m}\}$;
  end if
end for
if $\mathcal{U}_m = \phi$ **then**
  Set $\mathcal{Q}_m \leftarrow \mathcal{Q}_m \cap \mathcal{Q}_m \setminus \{\mathcal{V}_m[0]\}$; and repeat the above procedures by using Algorithms 3 and 4;
else
  sort$\{\eta_{n,m} \in \mathcal{U}_m\}$ in a descending order;
  for $\eta_{n,m} \in \mathcal{U}_m$ **do**
    $n \leftarrow \arg_{n \in \mathcal{N}} \eta_{n,m}$;
    if $\Upsilon_{n,m} \neq \phi$ and $\Psi_{n,l} > \Psi_{n,m}$ **then**
      if $h\, n^m \geq s_{n,c}^{cl}$ **then**
        $x_{n,m,s} = 1$; $y_{n,m,s} = 0$; $\mathcal{Q}_m \leftarrow \mathcal{Q}_m \cap \mathcal{Q}_m \setminus \{n\}$;
      else
        $\mathcal{V}_m \leftarrow n$; $x_{n,m,s} = 0$; $y_{n,m,s} = 1$;
      end if
    else
      $y_{n,m,s} = 0$; $\mathcal{Q}_m \leftarrow \mathcal{Q}_m \cap \mathcal{Q}_m \setminus \{n\}$; $\mathcal{C}_m \leftarrow \mathcal{C}_m \cup \{n\}$;
    end if
  end for
  Set $\mathcal{V}_m \leftarrow \mathcal{V}_m \cap \mathcal{V}_m \setminus \{\mathcal{V}_m[0]\}$; $\mathcal{Q}_m \leftarrow \mathcal{Q}_m \cap \mathcal{Q}_m \setminus \{\mathcal{V}_m[0]\}$;
  if $\mathcal{V}_m \neq \phi$ **then**
    Repeat the above procedures by using Algorithms 4;
  else
    Repeat the above procedures by using Algorithms 3 and 4;
  end if
end if
until $\mathcal{Q}_m = \phi$;

- In order to further adjust the task offloading strategy, we propose Algorithm 5 to adjust the performance of the previous algorithm. From Algorithm 2 we can get the $\mathcal{W} = \{\Psi_1, \Psi_2 \ldots, \Psi_M\}$.
- Then we select the edges with the largest and smallest consumption from $\mathcal{W}$, that are $m_h \leftarrow \arg\max_{m \in \mathcal{M}} \Psi_m$ and $m_l \leftarrow \arg\min_{m \in \mathcal{M}} \Psi_m$. We continue to adjust the offloading strategy through 6-12 steps until the conditions are met.

We analyze the time complexity of each algorithm. From the above description of all algorithms, we can easily get that the time complexity of Algorithm 1 is O(N^2M), the time complexity of RROS algorithm is O($M^2(M^2+N^2)$), the time complexity of Algorithm 3 is O(N^2), the time complexity of Algorithm 4 is O($M(M^2+N^2)$) and the time complexity of offloading strategy adjustment algorithm is O ($M^3(M^2+N^2)$). Since the number of users N is much larger than the number of MEC servers M, the time complexity of our offloading strategy can be calculated as O(N^2). Otherwise, if M and N are of the same magnitude or M is much larger than N, the time complexity of the algorithm will increase significantly.

Algorithm 5. Offloading Strategy Adjustment Algorithm

Obtain $\mathcal{W}$, $\mathcal{L}_m$, $\mathcal{C}_m$ by using Algorithm 2;
for $\Psi_m \in \mathcal{W}$ **do**
  $m_h \leftarrow \arg\max_{m \in \mathcal{M}} \Psi_m$; $m_l \leftarrow \arg\min_{m \in \mathcal{M}} \Psi_m$
end for
Calculate variance V_m of $\Psi_m \in \mathcal{W}$;
while $V_m >= Z$ **do**
  if $\mathcal{C}_{m_h} \neq \phi$ **then**
    $n \leftarrow rand\{n \in \mathcal{C}_{m_h}\}$; $K_{m_h} \leftarrow K_{m_h} \backslash \{n\}$; $K_{m_l} \leftarrow K_{m_l} \cup \{n\}$;
  else
    $n \leftarrow rand\{n \in \mathcal{L}_{m_h}\}$; $K_{m_h} \leftarrow K_{m_h} \backslash \{n\}$; $K_{m_l} \leftarrow K_{m_l} \cup \{n\}$;
  end if
  Repeat the above procedures by using Algorithms 2 and 5;
end while

6 Performance Evaluation

In this section, we will verify the theoretical results and evaluate the performance of the proposed computation offloading strategy through simulations. Throughout this section, the parameters are set as follows: the amount of UE_n's data b_n is randomly set from 0.5Mb to 30Mb, the execution requirement of the computation task $w_n = 5900/8$ CPU cycles per bit, the weight of response time of UE_n is $\beta_n^t = \{1, 0, 0.9, 0.5, 0.8, 0.45\}$, the weight of energy of UE_n $\beta_n^e = 1 - \beta_n^t$, the computing power of mobile user f_n^L is randomly set from 0.5GHz to 1GHz, the communication bandwidth $B_m = 2$ Mb, the channel gain $g_{n,m}$ is randomly set from 10^{-10} to 10^{-8}, the channel noise $N_0 = -80$dBm, the local CPU execution power p_n^L is randomly set from 4w to 7w, the receive (transmission) power $p_{rn} = p_{tn} = 2$w, the inherent energy consumption of the mobile device transmitter $L_n = 0.1$, the computing power of MEC_m f_n^m is randomly set from 10GHz to 12GHz, Inter-arrival time between tasks $\mathcal{G} =$

TABLE 2
The Performance Table of Different MECs (M = 10)

Edge	Processing frequency	Number of users	Average time
1	10.00	17.91	20.36
2	10.30	18.26	20.33
3	10.50	18.20	20.25
4	10.80	19.27	20.66
5	11.00	19.85	20.96
6	11.30	20.74	21.19
7	11.50	20.41	20.75
8	11.80	20.97	20.50
9	12.00	22.23	21.29
10	12.30	22.16	21.14

5. the communication rate of MEC_m offloading task to CC in round-trip $R_{ec} = 10$Mbps, the computing capability of CC for offloading task $f_n^c = 20$GHz, the amount of result data D_n is randomly set from $b_n/40$ bit to $b_n/80$ bit.

- For simplicity, we refer to our proposed computation offloading strategy as RROS. To evaluate the efficiency of RROS, we compare it with other four offloading strategies in the following.
- *Response ratio without CC offloading strategy(RR W/O CC)*: This offloading strategy is mostly similar to RROS, except that CC does not perform in the offloading model.
- *First come first served offloading strategy(FCFSOS)*: The offloading task framework is the same as the framework of RROS, with both MEC and CC participating in the offloading model. The difference is that this policy treats tasks in a firstcome-first-served fashion.
- *First come first served without CC offloading strategy (FCFSOS W/O CC)*: The offloading task framework is the same as RR W/O CC, CC does not participate in the offloading model. And this offloading strategy transactions with tasks in a first-come-first-served fashion.
- *SHAREAPP*: The offloading method is based on a three-tier frame with multiple users, one MEC server and single cloud center [32].

Table 2 shows the number of users and the average task completion time of different MEC servers when the number of MEC servers M is fixed ($M = 10$). As mentioned earlier, because every user has a MEC-choosing preference, the number of users of different MEC services will be different, even in the same communication situation. And the processing frequency of each MEC will be different. Obviously, as the processing frequency of MEC increases, so does the number of users. Also, it can be seen that as the processing frequency of MEC increases, the average time to complete tasks on each MEC increases. However, the growth rate of task completion average time is much smaller than the growth rate of the number of users. This is because in each MEC, we adopted the RROS algorithm to adjust the order of task execution, balance the task globally, and further optimize the completion time of all tasks.

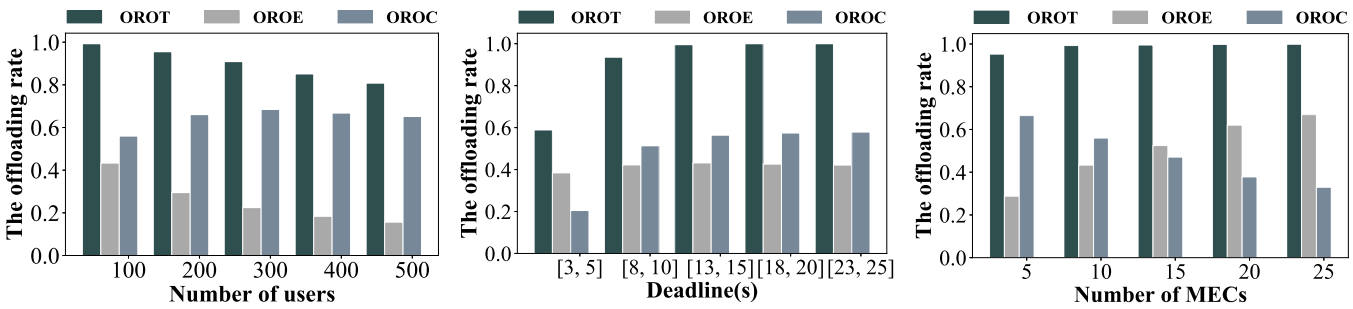
In order to verify the performance of the proposed RROS algorithm, we compared the offloading rate of different users, task execute time limit and the number of MECs. Here, we divide the offloading rate into three scenarios: the MEC offloading rate(OROE), the cloud server offloading rate(OROC), the total offloading rate including MECs and cloud center (OROT). As shown in Fig. 2a, as the number of users increases from 100 to 500, OROE and OROT gradually decrease, while OROC shows a trend of increasing first and then decreasing. This is because, when the number of users is relatively small, the number of users waiting on the MEC server is small, and MEC servers can provide service meets users quality constraints. As the number of users increases, the MEC server resources are insufficient to meet the quality constraints of users. Therefore, some tasks are offloaded to cloud center, and the OROC increases. Furthermore, as the number of tasks waiting on the MEC keep on increasing, the quality of service of many tasks cannot be met (for example, the wait time of a task exceeds its deadline) and these tasks would no longer be offloaded to the cloud center. The OROC begins to decline respectively.

Fig. 2b shows how the three offloading rates vary with user quality requirements (we set as task execute time deadline d_n). $N = 100$, $M = 10$, deadline $d_n \in [3, 5], [8, 10], [13, 15], [18, 20], [23, 25]$, evenly distributed. Definitely, OROE increases with dn and soon stabilizes. OROT and OROC also increase with the increase of d_n, and eventually tend to be stable. The reason for this is that with less time constraints on task completion, the number of tasks waiting at MECs will increase. However, offloading tasks to the cloud center reduces the total time for tasks to execute, so a large number of tasks change to offload to the cloud center for execution rather than waiting consistently on the MECs. Therefore, when the number of users reaches a certain level, OROE is stable, while OROC continues to increase and eventually it stabilizes.

A series of experiments are conducted to verify the offloading rate variation with the number of MECs in the network, as shown in Fig. 2c. Of course, as the number of MECs increases, OROE increases and OROC decreases. The number of tasks waiting on the MEC server also decreases as the number of MECs increases.

Next, in order to verify the performance of the proposed algorithm, we compare it with other algorithms. For simplicity, in subsequent experiments, we adopt the total offloading rate (that is, including the offloading rate of MECs and cloud center) as the performance evaluation indexes. Fig. 3 shows the offloading rates of different algorithms vary as the number of users. It can be seen from the figure, the offloading rates of all algorithms decrease with the increase of the number of users. Obviously, the algorithm proposed in this paper RROS has the smallest reduction. The offloading rate of RROS is significantly higher than that of RROSW/O CC. The result of FCFSOS compared with FCFSOS W/O CC is the same. This is because both RROS and FCFSOS apply three-tier heterogeneous service system, which makes full use of the computing resources of the cloud center. The MECs collaborate with the cloud center to jointly offload tasks, which will greatly reduce the number of waiting tasks on the MECs. In addition, when the number of users is 500, the offloading rate of RROS is 16.15% higher than that of FCFSOS and 67.56 % higher than that of FCFSOS W/O CC. The offloading rate of RROS W/O CC is also higher than FCFSOS W/O CC. The improvement of the performance is brought by the task scheduling policy proposed in this paper.

Fig. 4 shows the offloading rate of the four algorithms for different task execute deadline. It can be seen that with the



(a) Comparison of the offloading rate with number of users (b) Comparison of the offloading rate with deadline (c) Comparison of the offloading rate with number of MECs

Fig. 2. Comparisons of OROT, OROE, OROC with RROS.

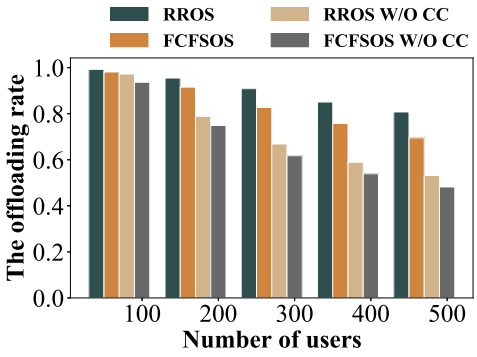


Fig. 3. The relationship between the offloading rate and number of users with different algorithms.

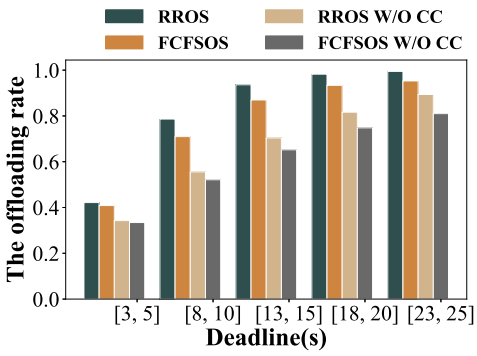


Fig. 4. The relationship between the offloading rate and task execute deadline with different algorithms.

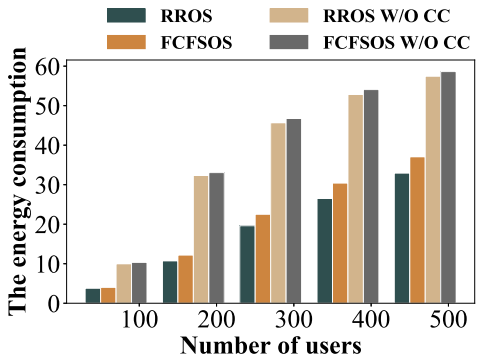


Fig. 5. Comparisons of the energy consumption with different algorithms.

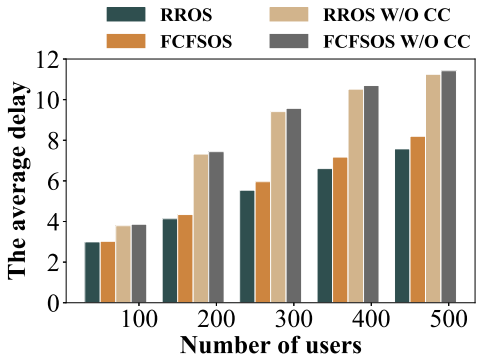


Fig. 6. Comparisons of the average delay with different algorithms.

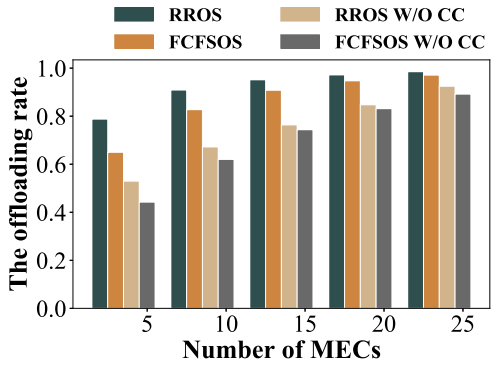


Fig. 7. Comparisons of the offloading rate with different algorithms.

increase of deadline, the offloading rates of these four strategies show an upward trend. But under the same deadline, RROS has the highest offloading rate. Whether compared to RROS W/O CC or FCFSOS, as the deadline increases, RROS has less advantage in offloading rate. This is because when the deadline is large enough, the optimization effect of task scheduling on the MEC is reduced.

Figs. 5 and 6 show the average power consumption and task execution latency of the four algorithms with different user numbers, respectively. It can be seen that the delay and energy consumption of these four offloading strategies increase with the increase of the number of users. But the growth rate of RROS is the lowest. In addition, when the number of users is constant, RROS has the lowest average delay and energy consumption among the four algorithms.

Fig. 7 shows the offloading rates of the four algorithms varying with the number of MECs. As the number of MECs increases, the offloading rates of all algorithms increase. RROS still has the highest offloading rate among the four algorithms on the same number of MECs. Furthermore, as the number of MEC increases, the differences of offloading rate among different algorithms decrease. This is because as the number of MECs increases, the number of tasks waiting

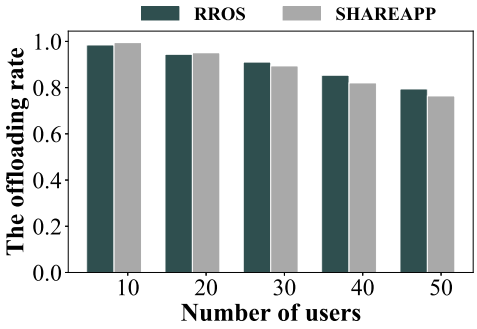


Fig. 8. Comparisons of the offloading rate between RROS and SHAREAPP.

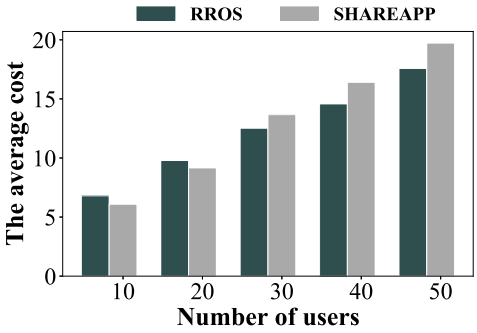


Fig. 9. Comparisons of the average cost between RROS and SHAREAPP.

on each MEC decreases. When the number of MECs is large enough, the number of tasks that need to be scheduled is small until it reaches zero.

All the above experiments consider resource collaboration among multiple MECs. Since the SHAREAPP algorithm only involves one computing access point, we further compare the performance of the RROS and the SHAREAPP algorithms. Accordingly, we analyze one MEC in the RROS strategy.

It can be seen from Figs. 8 and 9 that, with the increase of the number of users, the offloading rates of both RROS and SHAREAPP are decreasing and the average overhead are increasing. Here, the average cost refers to the combination of average delay and energy consumption. When the number of users is constant, the offloading rate of RROS is slightly higher than that of SHAREAPP, and the average cost of RROS is slightly lower than that of SHAREAPP. There are two reasons for this. On the one hand, SHAREAPP mainly adopts semi-positive relaxation. As the number of users increases, the performance of the algorithm to seek the optimal solution starts to decline. On the other hand, RROS algorithm includes the process of task scheduling and adjustment. When the number of users increases, the effectiveness of scheduling multiple tasks in a very short time is more significant.

7 Conclusion

We study a complex computing offloading problem in a three-tier frame with multiple users, multiple MECs, and a single cloud in this paper. Not only user preference but also the real-time nature of tasks is considered. In order to optimize the offloading decision of multiple users, we propose a RROS algorithm based on response ratio. The algorithm includes initial assignment based on user preference and dynamic adjustment mechanism among multiple MECs. Multiple MECs and the cloud center collaborate for all tasks from users. The experimental results demonstrate the effectiveness of the proposed strategy.

References

[1] P. Lai *et al.*, "Cost-effective app user allocation in an edge computing environment," *IEEE Trans. Cloud Comput.*, to be published, doi: 10.1109/TCC.2020.3001570.

[2] T. Soyata, R. Muraleedharan, C. Funai, M. Kwon, and W. Heinzelman, "Cloud-vision: Real-time face recognition using a mobile-cloudlet-cloud acceleration architecture," in *Proc. IEEE Symp. Comput. Commun.*, 2012, pp. 000 059–000 066.

[3] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empir. Methods Natural Lang. Process. Syst. Demonstrations*, 2020, pp. 38–45.

[4] M. Gerla, E.-K. Lee, G. Pau, and U. Lee, "Internet of vehicles: From intelligent grid to autonomous cars and vehicular clouds," in *Proc. IEEE World Forum Internet Things.*, 2014, pp. 241–246.

[5] N. Sharghivand, F. Derakhshan, L. Mashayekhy, and L. M. Khanli, "An edge computing matching framework with guaranteed quality of service," *IEEE Trans. Cloud Comput.*, to be published, doi: 10.1109/TCC.2020.3005539.

[6] A. Gandhi, P. Dube, A. Karve, A. P. Kochut, and L. Zhang, "Providing performance guarantees for cloud-deployed applications," *IEEE Trans. Cloud Comput.*, vol. 8, no. 1, pp. 269–281, Jan.–Mar. 2020.

[7] A. Alnoman, G. H. Carvalho, A. Anpalagan, and I. Woungang, "Energy efficiency on fully cloudified mobile networks: Survey, challenges, and open issues," *IEEE Commun. Surv. Tut.*, vol. 20, no. 2, pp. 1271–1291, Apr.–Jun. 2018.

[8] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.

[9] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, "On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1657–1681, Jul.–Sep. 2017.

[10] M. Agiwal, A. Roy, and N. Saxena, "Next generation 5G wireless networks: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 18, no. 3, pp. 1617–1655, Jul.–Sep. 2016.

[11] Y. Yu, "Mobile edge computing towards 5G: Vision, recent progress, and open challenges," *China Commun.*, vol. 13, pp. 89–99, 2016.

[12] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st Ed. MCC Workshop Mobile Cloud Comput.*, 2012, pp. 13–16.

[13] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 587–597, Mar. 2018.

[14] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surv. Tut.*, vol. 19, no. 4, pp. 2322–2358, Oct.–Dec. 2017.

[15] M. Du, Y. Wang, K. Ye, and C.-Z. Xu, "Algorithmics of cost-driven computation offloading in the edge-cloud environment," *IEEE Trans. Comput.*, vol. 69, no. 10, pp. 1519–1532, Oct. 2020.

[16] "Ericsson mobility report, june 2020," Accessed: Jun. 7, 2020. [Online], Available: https://www.ericsson.com/en/mobility-report/reports

[17] C. Wang, C. Liang, F. R. Yu, Q. Chen, and L. Tang, "Computation offloading and resource allocation in wireless cellular networks with mobile edge computing," *IEEE Trans. Wirel. Commun.*, vol. 16, no. 8, pp. 4924–4938, Aug. 2017.

[18] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1628–1656, Jul.–Sep. 2017.

[19] Y. Qu *et al.*, "Robust offloading scheduling for mobile edge computing," *IEEE Trans. Mobile Comput.*, to be published, doi: 10.1109/TMC.2020.3043000.

[20] R. S. Sanketh, Y. MohanaRoopa, and P. V. N. Reddy, "A survey of fog computing: Fundamental, architecture, applications and challenges," in *Proc. 3rd Int. Conf. I-SMAC*, 2019, pp. 512–516.
[21] C. You, K. Huang, H. Chae, and B. Kim, "Energy-efficient resource allocation for mobile-edge computation offloading," *IEEE Trans. Wireless Commun.*, vol. 16, no. 3, pp. 1397–1411, Mar. 2017.
[22] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.
[23] J. Zhang *et al.*, "Energy-latency tradeoff for energy-aware offloading in mobile edge computing networks," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2633–2645, Aug. 2018.
[24] T. Q. Dinh, J. Tang, Q. D. La, and T. Q. S. Quek, "Offloading in mobile edge computing: Task allocation and computational frequency scaling," *IEEE Trans. Commun.*, vol. 65, no. 8, pp. 3571–3584, Aug. 2017.
[25] S. Guo, B. Xiao, Y. Yang, and Y. Yang, "Energy-efficient dynamic offloading and resource scheduling in mobile cloud computing," in *Proc. IEEE INFOCOM 35th Annu. IEEE Int. Conf. Comput. Commun.*, 2016, pp. 1–9.
[26] Y. Mao, J. Zhang, S. H. Song, and K. B. Letaief, "Power-delay tradeoff in multi-user mobile-edge computing systems," in *Proc. IEEE Glob. Commun. Conf.*, 2016, pp. 1–6.
[27] L. Yang, J. Cao, Z. Wang, and W. Wu, "Network aware multi-user computation partitioning in mobile edge clouds," in *Proc. Int. Conf. Parallel Process.*, 2017, pp. 302–311.
[28] M. Chen, M. Dong, and B. Liang, "Resource sharing of a computing access point for multi-user mobile cloud offloading with delay constraints," *IEEE Trans. Mobile Comput.*, vol. 17, no. 12, pp. 2868–2881, Dec. 2018.
[29] M. R. Rahimi, N. Venkatasubramanian, S. Mehrotra, and A. V. Vasilakos, "MAPcloud: Mobile applications on an elastic and scalable 2-tier cloud architecture," in *Proc. IEEE 5th Int. Conf. Utility Cloud Comput.*, 2012, pp. 83–90.
[30] L. Liu, Z. Chang, X. Guo, S. Mao, and T. Ristaniemi, "Multiobjective optimization for computation offloading in fog computing," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 283–294, Feb. 2018.
[31] M.-H. Chen, B. Liang, and M. Dong, "A semidefinite relaxation approach to mobile cloud offloading with computing access point," in *Proc. IEEE 16th Int. Workshop Signal Process. Adv. Wireless Commun.*, 2015, pp. 186–190.
[32] M.-H. Chen, M. Dong, and B. Liang, "Joint offloading decision and resource allocation for mobile cloud with computing access point," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2016, pp. 3516–3520.
[33] W. Labidi, M. Sarkiss, and M. Kamoun, "Joint multi-user resource scheduling and computation offloading in small cell networks," in *Proc. IEEE 11th Int. Conf. Wireless Mobile Comput., Netw. Commun.*, 2015, pp. 794–801.
[34] K. Li, "Optimal task execution speed setting and lower bound for delay and energy minimization," *J. Parallel Distrib. Comput.*, vol. 123, pp. 13–25, 2019.
[35] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.

Shujuan Tian received the PhD degree from Central South University in 2016. From September 2019 to September 2020, she was a postdoctoral researcher with the Computer Science Department, Illinois Institute of Technology (IIT), and cooperated with Professor Pengjun Wan. She is currently an associate professor with the School of Computer Science and School of Cyberspace Security, Xiangtan University. Her research interests include mobile edge computing, VANET, Internet of Things, and wireless signal control and processing, etc.

Chi Chang is currently working toward the graduation degree with the School of Automation and Information, Xiangtan University. His research interests include task scheduling and offloading in edge computing, and the application of machine learning in mobile edge computing.

Saiqin Long received the PhD degree in computer applications technology from the South China University of Technology in 2014. She is currently an associate professor with the School of Computer Science, Xiangtan University. Her research interests include cloud computing, cloud storage, parallel and distributed systems, file systems, and computer system architecture. She is a member of Chinese Computer Federation.

Sangyoon Oh received the PhD degree from the Computer Science Department, Indiana University Bloomington, Bloomington, IN, USA. He is currently a professor with the School of Information and Computer Engineering, Ajou University, South Korea. Before joining Ajou University, he was with SK Telecom, South Korea. His research interests mainly include parallel and distributed computing, high performance computing, distributed deep learning, and graph data processing.

Zhetao Li (Member, IEEE) received the BEng degree in electrical information engineering from Xiangtan University in 2002, the MEng degree in pattern recognition and intelligent system from Beihang University in 2005, and the PhD degree in computer application technology from Hunan University in 2010. He is currently a professor with the College of Computer, Xiangtan University. From Dec 2013 to Dec 2014, he was a postdoctoral in wireless network with Stony Brook University. He is a member of CCF.

Jun Long received the MSc and PhD degrees from Central South University in 2003 and 2011, respectively. From 2012 to 2013, he was a visiting professor with Columbia University. He is currently a professor with the School of Information Science and Engineering of Central South University, China. His research interests include network management, Qos guarantees, web service, and multimedia systems and retrieval.